FASTT: Feature-Adaptive Speech-to-Text Model Selection

***Abstract*—**

***Index Terms*—**

I. Introduction

Automatic speech recognition (ASR) has advanced rapidly with the emergence of large-scale pretrained models and self-supervised representation learning [1], [2], [3], [4]. Contemporary speech-to-text (STT) systems such as transformer encoder–decoder architectures and transducer-based models achieve strong performance across diverse speech domains [5], [6], [7]. Nevertheless, despite their scale and robustness, no single STT model consistently achieves optimal transcription accuracy across all acoustic conditions [1], [8].

In real-world deployments, audio inputs demonstrate significant variability in noise characteristics, channel properties, recording devices, speaker identity, accent, speaking rate, and articulation style [8]. This heterogeneity induces model-dependent performance variation. Different STT systems often demonstrate complementary strengths and weaknesses. A model optimized for accented or conversational speech may underperform in noisy telephony recordings, while another tuned for clean read speech may degrade under spontaneous dialogue or reverberant environments [8]. Consequently, the identity of the best-performing STT model can vary across audio inputs, which motivates the problem of adaptive model selection [9], [10]. Rather than committing to a single fixed STT model, one may aim to select, for each input, the model expected to yield the lowest transcription error. Traditional system-combination techniques such as ROVER [11] reduce error by fusing multiple hypotheses, but require executing multiple full decoders at inference time, increasing computational cost proportionally to the number of experts. In contrast, adaptive selection seeks to exploit model diversity while introducing only minimal computational overhead beyond a single model run.

A fundamental difficulty in adaptive selection lies in reliably predicting expert performance from input-dependent characteristics. Modern acoustic representations often integrate low-level spectral features, prosodic statistics, signal-quality measures, and high-dimensional neural embeddings [12], [2]. While expressive, such feature spaces may contain substantial redundancy, correlated dimensions, and weakly informative components especially when the amount of labeled selection data is limited. In this regime, directly learning a selector over the raw feature space can lead to overfitting, unstable selection behavior, and poor generalization across acoustic domains. The key insight underlying this work is that selection quality depends not only on the selection architecture, but also on the geometry of the feature space in which selection decisions are made. Here, we introduce FASTT, a feature-adaptive selection framework that learns a task-aligned feature transformation together with a selection function under a cumulative transcription-error objective. By explicitly adapting the feature representation to the selection loss, and the selection function in an end to end manner in order to optimize the performance our approach significantly improve generalization across heterogeneous acoustic conditions while maintaining computational efficiency at inference time.

FASTT is designed as an end-to-end framework that jointly learns task-aligned feature transformations and an adaptive STT selection policy. The core objective of the framework is to identify speech characteristics that distinguish the performance regimes of different speech-to-text (STT) systems and to exploit these characteristics to select the most suitable transcription algorithm for each input. Given a pool of pretrained STT models, FASTT trains a selector that predicts which STT system is expected to produce the lowest transcription error for a given audio input. The selector operates on features derived from the underlying speech signal, including easily extractable acoustic descriptors and semantic embeddings. Through the learnable feature transformation module, the framework constructs a task-adaptive feature representation that emphasizes the speech attributes most informative for discriminating between candidate STT systems. In this sense, FASTT learns a specialized STT-selection policy tailored to the specific ensemble of transcription algorithms used in the system. During training, the framework learns the parameters of both the feature transformation and the selector using a reference dataset that captures diverse acoustic conditions. The objective is to minimize the cumulative transcription error across the dataset by learning to route each audio input to the STT model expected to perform best. Once trained, the selector evaluates the feature representation of each incoming audio input and selects a single STT model for transcription. Because only one expert is executed at inference time, the computational cost remains comparable to that of a single-model system. Unlike conventional approaches that rely on static model selection or computationally expensive ensemble combination techniques such as ROVER [11], FASTT performs input-adaptive STT selection using a lightweight learned selector. This allows the system to exploit complementary strengths of different STT models while maintaining efficient inference. Furthermore, we provide convergence guarantees that characterize the stability of the learning procedure under mild regularity conditions.

A. Related Work

Prior work has focused on improving ASR robustness under acoustic variability, including noise, channel distortion,

reverberation, and speaker diversity [1], [8]. Techniques such as multi-condition training, data augmentation [13], domain adaptation [14], and large-scale self-supervised pretraining [2], [3] have improved model generalization across diverse environments. These approaches aim to train a single model that performs well across conditions by increasing representational capacity and data coverage. In contrast, we assume a fixed pool of pretrained STT models and address the complementary problem of selecting, rather than retraining, the most suitable expert for each input.

Traditional ensemble methods aim to reduce transcription error by aggregating hypotheses from multiple ASR models. An example approach is Recognizer Output Voting Error Reduction (ROVER) [11], which aligns hypotheses via word-level alignment and performs majority voting. Subsequent extensions include confusion-network alignment and consensus decoding [15], lattice-based combination techniques [16], and minimum Bayes risk system combination frameworks [17]. More recent neural approaches have explored posterior averaging, logit-level fusion, and ensemble decoding strategies across independently trained ASR models [18], [6]. While these techniques often yield measurable WER reductions, they require executing multiple full ASR decoders for every utterance, followed by alignment, fusion, or rescoring steps. As a result, inference complexity scales approximately linearly with the number of experts, making such approaches computationally expensive in large-scale or latency-sensitive deployments. In contrast, our framework performs input-adaptive model selection rather than hypothesis fusion. Only a single pretrained STT expert is executed at inference time, and selection decisions are computed via a lightweight selection function operating on pre-extracted features. Consequently, the computational overhead relative to single-model deployment remains minimal while still exploiting performance diversity across experts.

Adaptive expert selection is conceptually related to mixture-of-experts (MoE) models [19], [20], in which a gating network assigns input-dependent weights to multiple expert components. Classical MoE architectures jointly train both the experts and the gating function within a single end-to-end model, typically sharing intermediate representations and optimizing a unified likelihood objective. More recent sparse and conditional computation frameworks, such as sparsely gated MoE layers and Switch Transformers [20], [21], dynamically activate subsets of parameters to improve scalability and computational efficiency. However, our setting differs fundamentally from standard MoE formulations in several respects. First, the experts in our framework are independently pretrained, large-scale STT systems whose parameters remain fixed during selector training. There is no shared backbone, parameter sharing, or joint optimization between the experts and the selection mechanism. This design allows the framework to incorporate arbitrary pretrained STT systems without requiring additional joint training or fine-tuning, making it easy to deploy with heterogeneous transcription models. Second, the selection objective is not maximum likelihood or classification accuracy within a unified model, but cumulative transcription-error minimization across heterogeneous acoustic conditions. This is important because the ultimate goal of the system is to reduce transcription error in practice, rather than optimizing a surrogate objective unrelated to the final evaluation metric. Third, selection occurs at the system level selecting among full, independently deployed ASR models rather than activating subnetworks within a single architecture. Consequently, we study supervised expert selection as a meta-learning problem over frozen high-capacity models, rather than as a jointly trained mixture model. This design avoids costly joint training and allows the framework to operate with arbitrary pretrained STT systems without modifying their parameters.

A related line of work considers confidence-based and quality estimation (QE) approaches for ASR output selection and routing. Posterior probability–based confidence scores can guide rescoring, hypothesis selection, or fallback decisions [16]. Domain classification and environment detection have also been explored for routing utterances to domain-specialized models [22]. However, confidence-based approaches require at least partial decoder execution to compute posterior scores, while domain classifiers only capture coarse categorical distinctions. In contrast, FASTT operates entirely on pre-extracted acoustic features without executing any STT decoder at selection time, and learns fine-grained, continuous selection distributions that capture subtle performance differences across experts.

Feature selection and representation learning within tree-based and boosting frameworks have been studied through differentiable decision trees and soft selection architectures [23], [24]. In particular, recent work on soft gradient boosting with learnable feature transforms for sequential regression [25] demonstrates that jointly optimizing feature transformations and boosted soft decision trees can improve regression performance in high-dimensional settings by aligning feature geometry with the learning objective. Our work extends this principle to the expert-selection problem in speech recognition. Instead of minimizing a scalar regression loss, we optimize a cumulative transcription-error objective over a probability simplex of pretrained STT experts. Moreover, rather than modeling sequential regression targets, we address input-dependent expert performance prediction under heterogeneous acoustic conditions. Our framework combines task-aligned feature adaptation with supervised selection to reduce redundancy in high-dimensional acoustic and embedding features, while remaining stable even when the selector is trained with limited data.

B. Contributions

Our contributions are summarized as follows:

- We introduce FASTT, a general-purpose framework for adaptive STT model selection that jointly learns a task-aligned feature transformation and a selection network under a cumulative transcription-error objective. The framework selects the most suitable STT model for each audio input with minimal computational overhead beyond a single model execution.
- We present a principled approach for feature adaptation within the framework, including diagonal gating,

linear, low-rank, and nonlinear transformation variants with sparsity and stability regularization. These adaptations improve selection reliability in high-dimensional and data-limited settings.
- We provide two optimization paths for joint training of the feature transformation and the selector: (i) end-to-end backpropagation for differentiable selectors, using a boosted architecture inspired by [25], and (ii) alternating optimization for non-differentiable selectors such as gradient-boosted trees.
- The framework is loss-agnostic (any transcription quality metric such as WER or CER can be used), feature-agnostic (any set of discriminative audio features can be used), and selector-agnostic (any learning algorithm can serve as the selection function).
- We provide our implementation as open-source code to support reproducibility and enable researchers to extend the framework with custom features, selectors, and expert models.

C. Organization

The remainder of the paper is organized as follows. Section II formalizes the supervised expert-selection problem. Section III presents the proposed feature-adaptive selection framework, including the learnable transformation and the selection function. Section IV describes the experimental setup and empirical evaluation. Finally, Section V concludes the paper.

II. Problem Formulation

We consider the problem of adaptive speech-to-text (STT) model selection for a collection of audio input $\{\mathbf{s}_i\}_{i=1}^N$, where N may be large and the acoustic characteristics may vary substantially across inputs. Each input can demonstrate diverse properties, including noise type and level, channel conditions, speaker identity, accent, speaking rate, and articulation variability, which can significantly affect the transcription accuracy of different STT systems [1], [8].

Let $\mathcal{E} = \{E_1, E_2, \ldots, E_L\}$ denote a pool of L pretrained STT experts. For an input utterance $\mathbf{s}_i$ and expert E_j, the produced transcription is written as $y_{i,j} = E_j(\mathbf{s}_i)$. Given the reference transcription y_i^{ref}, the quality of the produced transcript is measured using an error function

$$\ell(y_{i,j},\, y_i^{\text{ref}}) \in [0, 1],$$

which may correspond to word error rate (WER), character error rate (CER), or another normalized transcription metric [26].

In practice, no single STT model is uniformly optimal across heterogeneous acoustic conditions [1], [4], [8]. Instead, the optimal model achieving the lowest transcription error may vary from one utterance to another depending on the underlying speech characteristics of the input s_i. This motivates the formulation of an expert selection problem in which the system must decide which expert should process each input utterance. Let each input utterance $\mathbf{s}_i$ be represented by a feature vector $\mathbf{z}_i \in \mathbb{R}^p$ computed using a fixed feature-extraction pipeline. A routing policy is defined as a function $\pi : \mathbb{R}^p \to \{1, \ldots, L\}$, which assigns an expert index to each feature vector. The transcription loss incurred by a routing policy π over the dataset is defined as

$$J(\pi) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(y_{i,\pi(\mathbf{z}_i)},\, y_i^{\text{ref}}\big)\,.$$

The objective of adaptive expert selection is to identify the optimal routing policy within a class of admissible policies Π:

$$\pi^\dagger = \arg\min_{\pi \in \Pi} J(\pi).$$

The main challenge in solving this problem arises from the potentially high dimensionality of the feature representation and the limited availability of labeled training examples for learning the routing policy. Furthermore, the relative performance of different STT experts may depend on subtle variations in the acoustic properties of the speech signal. Next we introduce a learning-based routing framework that estimates an input-dependent distribution over the available STT experts. The framework first learns a task-adaptive transformation of the feature representation and then applies a selection model that predicts expert suitability. The final transcription is obtained by selecting the expert with the highest predicted suitability for the given input.

III. FASTT: Feature Adaptive Speech-to-Text Model Selection

A. Algorithm Overview

The algorithm consists of two phases: an offline training phase and an online inference phase, as illustrated in Fig. 1. In the offline phase, the system learns how to predict the relative suitability of different STT experts based on input-dependent audio characteristics. For each labeled training input s_i, the framework extracts a feature vector $\mathbf{z}_i$ using the feature pipeline described in Section III-B. This feature representation includes acoustic statistics, prosodic descriptors, signal-level measures, and neural speech embeddings aggregated at the utterance level. To improve selection reliability in high-dimensional feature spaces, the framework learns a parametric feature transformation that maps the original feature vector $\mathbf{z}_i$ into a task-aligned representation as shown in Fig. 1. This transformation reshapes the feature space to better capture the relationship between acoustic characteristics and expert performance. The transformed features are then passed to a selection function that produces a weight vector over the candidate STT experts $\{E_1, \ldots, E_L\}$. The parameters of the feature transformation and the selection function are learned during the offline phase using evaluations of all experts on labeled audio clips, minimizing the cumulative transcription loss defined in Section II.

In the online inference phase, given a new input audio s_i, the system first extracts the feature vector $\mathbf{z}_i$ as shown in Fig. 1. The trained selection model then computes a weight vector $\mathbf{w}_n$ over the candidate experts based on the transformed feature representation. A single expert E_{i_j} is selected according to the highest predicted probability, and the final transcription y_{i,E_i} is

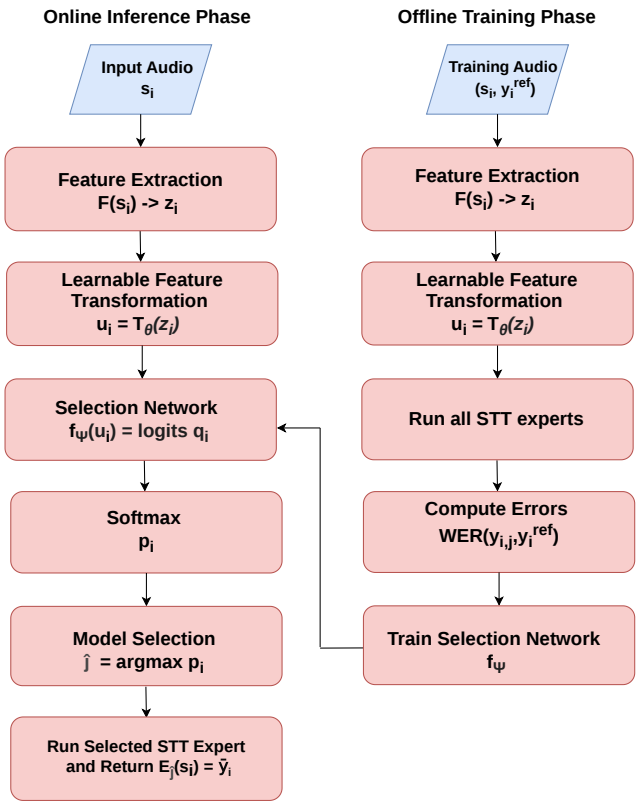

Fig. 1. Overview of the FASTT framework. The offline phase extracts acoustic features, applies a learnable feature transformation, and trains the selection function using expected-WER minimization. At inference, only the selected STT expert is executed, introducing minimal overhead beyond a single model run.

Algorithm 1 Online STT Model Selection via FASTT

Inputs:

- Unlabeled input audio clip s_i
- Set of audio feature extraction functions $\mathbb{F}(\cdot)$
- Trained FASTT selection pipeline f_{FASTT}
- STT expert pool $\mathcal{E} = \{E_1, \ldots, M_L\}$

Outputs:

- Final transcription $\widehat{y}_i$ for audio s_i

Inference Steps:

1: Extract utterance-level features: $\mathbf{z}_i \leftarrow \mathbb{F}(s_i)$
2: Apply the trained FASTT pipeline to compute selection logits: $\mathbf{q}_i \leftarrow f_{\text{FASTT}}(\mathbf{z}_i)$
3: Normalize $\mathbf{q}_i$ via softmax to obtain weights $\mathbf{p}_i$
4: Select expert model:

$$j_i \leftarrow \arg \max_{j \in \{1,\ldots,K\}} p_{i,j}$$

5: Return final transcription: $\hat{y}_i \leftarrow E_{\hat{j}}(s_i)$

obtained by executing only that expert. Because only one STT model is executed at inference time, the framework introduces only minimal computational overhead beyond a single model run while still exploiting performance diversity across multiple pretrained experts. The complete online inference procedure is summarized in Algorithm 1, and the full pipeline is illustrated in Fig. 1.

B. Feature Representation

For each incoming audio input s_i, we construct a compact feature vector $\mathbf{z}_i \in \mathbb{R}^p$ to encode acoustic and contextual information relevant to STT performance as shown in Fig. 1. These features summarize signal quality, phonetic content, speaker characteristics, and temporal dynamics [12], [27].

1) Spectral–Phonetic Features: We extract 13 Mel-frequency cepstral coefficients (MFCCs) together with their first- and second-order derivatives (Δ and $\Delta\Delta$) [28], [29]. Lower-order coefficients represent the coarse spectral envelope, while higher-order terms encode finer phonetic detail. The inclusion of temporal derivatives provides robustness to coarticulation, prosodic variation, and speaker-specific articulation [12].

2) Prosodic Features: Prosodic features, including pitch statistics (mean, range) and silence ratios derived from voice activity detection (VAD), characterize speaking rhythm, stress, and expressiveness [30]. Such features are particularly valuable for identifying conditions where transcription accuracy may degrade under expressive, emotional, or noisy speech [27].

3) Neural Speech Embeddings: High-level speech representations are extracted from pretrained self-supervised encoders such as Wav2Vec2 and HuBERT [2], [3]. These embeddings encapsulate rich information about phonetic identity, speaker traits, and semantic context, complementing conventional low-level features. Because they require only a single encoder forward pass without full decoding, they introduce negligible overhead relative to the STT inference itself. Specifically, the SSL encoder forward pass typically requires less than 50ms per utterance on modern GPU hardware, which is an order of magnitude smaller than the full ASR decoding time. For resource-constrained deployment scenarios, lighter embedding models or knowledge-distilled variants could be substituted without modifying the framework.

4) Signal and Temporal Features: Signal-level features including signal-to-noise ratio (SNR), zero-crossing rate (ZCR), and spectral entropy quantify acoustic clarity and spectral irregularity [31], [32]. Temporal features such as utterance duration, speech rate, and pause ratio measure fluency and pacing, both of which significantly impact STT robustness [13], [33]. The resulting feature vector $\mathbf{z}_i$ thus integrates spectral, prosodic, neural, and temporal features into a unified contextual representation for each input. This vector serves as the conditioning signal for the supervised selection function.

Remark 1: Our framework is feature-agnostic: The proposed framework does not assume or require any specific feature family. The selection network operates on a generic feature vector and can incorporate any set of discriminative features such as acoustic, linguistic, prosodic, model-intrinsic, or domain-specific features that correlate with relative STT performance. If alternative features provide better separation between candidate algorithms for a given application, they can be substituted or augmented without modifying the training procedure, loss function, or selection mechanism.

C. Learnable Feature Transformation

The feature representation $\mathbf{z}_i \in \mathbb{R}^p$ described in Section III-B captures diverse acoustic and embedding-based properties of the input audio. While such descriptors provide informative signals for expert selection, their raw representation may not be optimally structured for predicting relative expert performance. In particular, high-dimensional feature spaces may contain correlated dimensions or redundant information that can make the selection model more difficult to train when labeled selection data is limited.

Classical approaches to dimensionality reduction in such settings include filter-based methods (e.g., mutual information ranking, variance thresholding), wrapper-based strategies (e.g., sequential feature selection) [34], [35], and embedded methods that couple feature selection with model training [23]. However, filter methods disregard inter-feature dependencies and their interaction with the downstream objective, wrapper approaches have combinatorial search costs that scale poorly with p, and embedded methods are typically tied to specific model families. These limitations motivate a learnable, task-aligned feature transformation that can be jointly optimized with any downstream selector.

We introduce a parametric mapping applied prior to the selection network that reshapes the geometry of the feature space without modifying the feature extraction pipeline itself, as illustrated in Fig. 1. The transformed representation is defined as

$$\mathbf{u}_i = \mathbb{T}_\theta(\mathbf{z}_i),$$

where $\mathbb{T}_\theta : \mathbb{R}^p \to \mathbb{R}^{d'}$ denotes a parametric transformation with learnable parameters θ, and $\mathbf{u}_i$ serves as the input to the downstream expert selection function. The optimization strategy for θ depends on whether the downstream selector is differentiable, as discussed in Section III-E. We present several transformation variants below; all are compatible with any selector variant within the framework.

1) Diagonal Feature Gating: As a capacity-controlled transformation, we consider feature-wise scaling:

$$\mathbf{u}_i = \mathbf{q} \odot \mathbf{z}_i,$$

where $\mathbf{q} \in \mathbb{R}^p$ is a learnable gating vector and $\odot$ denotes element-wise multiplication. Each component q_j assigns an importance weight to the corresponding feature dimension j. This formulation acts as a differentiable feature-selection mechanism: dimensions with small q_j values are effectively suppressed, while informative features receive larger weights. Because the number of additional parameters is limited to d, the diagonal gating transformation provides a simple yet effective mechanism for controlling overfitting in high-dimensional, data-limited regimes. To promote sparsity and stability, we apply combined ℓ_1/ℓ_2 regularization:

$$\lambda_1 \|\mathbf{q}\|_1 + \lambda_2 \|\mathbf{q}\|_2^2.$$

2) Linear Feature Transformation: A more expressive alternative is a full linear projection:

$$\mathbf{u}_i = \mathbf{W}\mathbf{z}_i + \mathbf{b},$$

where $\mathbf{W} \in \mathbb{R}^{d' \times p}$ is a learnable projection matrix and $\mathbf{b} \in \mathbb{R}^{d'}$ is a bias vector. This transformation allows the model to reweight and mix the original feature dimensions in a task-aligned manner. By choosing $d' < p$, the transformation acts as a dimensionality-reduction mechanism. Weight decay regularization is applied to $\mathbf{W}$ to control overfitting.

3) Low-Rank Linear Transformation: While the full linear transformation allows arbitrary mixing of feature dimensions, it may introduce a large number of parameters when p is high. To provide a more capacity-controlled alternative, we consider a low-rank factorization of the projection matrix:

$$\mathbf{u}_i = \mathbf{W}_2 \mathbf{W}_1 \mathbf{z}_i,$$

where $\mathbf{W}_1 \in \mathbb{R}^{r \times p}$, $\mathbf{W}_2 \in \mathbb{R}^{d' \times r}$, and $r \ll \min(p, d')$ is a bottleneck dimension. This factorization constrains the rank of the projection, allowing the model to capture interactions between feature dimensions while reducing the number of learnable parameters.

4) Nonlinear Feature Transformation: To capture nonlinear feature interactions that may affect relative expert performance, we consider a nonlinear bottleneck transformation:

$$\mathbf{u}_i = \mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{z}_i),$$

where $\sigma(\cdot)$ is an element-wise activation function (e.g., ReLU or GELU), $\mathbf{W}_1 \in \mathbb{R}^{r \times p}$ and $\mathbf{W}_2 \in \mathbb{R}^{d' \times r}$ are learnable weight matrices, and $r \ll \min(p, d')$ is the bottleneck dimension. The bottleneck structure constrains the parameter count and helps prevent overfitting when labeled selection data is limited.

Remark 2: The proposed framework is flexible with respect to the choice of feature transformation. In addition to the variants presented here, other parametric transformations such as deeper neural encoders, kernel-based mappings, or representation learning modules could be used without modifying the overall pipeline. All transformation variants are evaluated in Section IV.

D. Selection Objective

Given the transformed feature vector $\mathbf{u}_i \in \mathbb{R}^{d'}$ obtained from the feature transformation stage described in Section III-C, the goal of the selection function is to assign preference scores to a set of L pretrained STT experts $\mathcal{E} = \{E_1, \ldots, E_L\}$. We define the selection function as a parametric mapping

$$f_\psi(\mathbf{u}_i) = \mathbf{q}_i \in \mathbb{R}^L,$$

where ψ denotes the trainable parameters and $\mathbf{q}_i = (q_{i,1}, \ldots, q_{i,L})$ represents the logits associated with the candidate STT experts. Each component $q_{i,j}$ reflects the relative suitability of expert E_j for the audio input s_j. We apply a softmax operation to the logits to obtain a normalized selection probability for each expert model:

$$p_{i,j} = \frac{\exp(q_{i,j})}{\sum_{m=1}^{L} \exp(s_{i,m})}, \qquad j = 1, \ldots, L, \tag{1}$$

where $p_{i,j}$ denotes the probability assigned to expert E_j. Given these probabilities, we define the expected transcription loss for the audio clip s_i as

$$\mathcal{L}_n = \sum_{j=1}^{L} p_{i,j} \, \mathrm{WER}(y_{i,j}, \, y_i^{\mathrm{ref}}), \tag{2}$$

where y_i^{ref} is the reference transcription. The WER between a hypothesis transcription $y_{i,j}$ and the reference y_i^{ref} is defined as $\mathrm{WER}(y_{i,j}, \, y_i^{\mathrm{ref}}) = \frac{S+I+D}{|\, y_i^{\mathrm{ref}} \,|}$, where S, I, and D denote the numbers of substitutions, insertions, and deletions obtained from a standard Levenshtein alignment [8].

Remark 3: Although we use WER as the optimization objective due to its widespread use and interpretability in speech recognition research [1], [8], other transcription quality metrics such as CER or task-specific error measures could also be used within the same framework.

The overall training objective minimizes the expected transcription error across the dataset:

$$\min_{\theta,\psi} \mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_n = \frac{1}{N} \sum_{n=1}^{N} \sum_{j=1}^{L} p_{i,j} \, \mathrm{WER}(y_{i,j}, \, y_i^{\mathrm{ref}}). \tag{3}$$

During training, the WER values $\mathrm{WER}(y_{i,j}, \, y_i^{\mathrm{ref}})$ are treated as fixed constants obtained from offline evaluation of the STT experts. The optimization strategy for the feature transformation parameters θ and the selection function parameters ψ depends on whether the selection function is differentiable with respect to its inputs and parameters, as discussed in the following subsection.

E. Joint Feature and Model Selection

A key contribution of the FASTT framework is the joint training of the feature transformation $\mathbb{T}_\theta$ and the selection function f_ψ. Instead of performing feature selection and model selection as two separate steps, FASTT learns both components together using the expected transcription error. This joint training aligns the feature representation with the model selection task.

The framework supports two optimization paths depending on the differentiability of the selection function.

1) FASTT for Differentiable Model Selectors: When the selection function $f_\psi(\cdot)$ is differentiable with respect to its parameters ψ and its inputs $\mathbf{u}_i$, the parameters θ and ψ are optimized jointly using standard backpropagation. Gradients of the expected word error rate (WER) objective flow from the softmax probabilities, through the selection network, and into the feature transformation. This strategy enables end-to-end feature adaptation in a single training loop without requiring any auxiliary steps.

As a representative differentiable model, we use Soft Decision Trees (SDTs). SDTs are hierarchical models where each internal node makes a soft, probabilistic routing decision based on a linear combination of features. Because these routing decisions are continuous rather than completely discrete, SDTs are fully differentiable and suitable for gradient descent. We construct our selector, FASTT-SDT, using an additive ensemble of SDTs, as in [25]. The model consists of B sequential boosting rounds. At each round b, a per-round feature transformation $\mathbb{T}_{\theta_b}(\cdot)$ is applied to the input features, followed by a soft decision tree $h_b(\cdot)$ providing L logits. The final output is the sum of logits across all boosting rounds:

$$\mathbf{q}_i = \sum_{b=1}^{B} h_b(\mathbb{T}_{\theta_b}(\mathbf{z}_i)) \, . \tag{4}$$

At each round, the current tree is fit on the gradient of the previous predictions. The per-round feature transformation $\mathbb{T}_{\theta_b}$ and the soft decision tree h_b are optimized jointly using backpropagation on the expected-WER objective from (3). A key property of BFS is that each boosting round learns its own feature transformation, allowing different rounds to focus on different subsets or combinations of the input features. The complete offline training procedure is given in Algorithm 2.

Algorithm 2 Offline Training: FASTT for Differentiable Selectors

Inputs:
- $\{(s_1, \, y_1^{\mathrm{ref}}), \ldots, (s_N, \, y_N^{\mathrm{ref}})\}$: Labeled audio clips with reference transcriptions
- $\mathbb{F}$: Feature extraction pipeline
- $\mathcal{E} = \{E_1, \ldots, E_L\}$: Pretrained STT expert pool
- B: Number of boosting rounds

Output: Trained FASTT model $\{(\mathbb{T}_{\theta_b}, h_b)\}_{b=1}^{B}$

Offline data preparation:
for $n \leftarrow 1$ **to** N **do**
  Extract features: $\mathbf{z}_i \leftarrow \mathbb{F}(s_i)$
  for each expert $E_j \in \mathcal{E}$ **do**
    Transcribe: $y_{i,j} \leftarrow E_j(s_i)$
    Compute: $\mathrm{WER}_{i,j} \leftarrow \mathrm{WER}(y_{i,j}, \, y_i^{\mathrm{ref}})$
  end for
end for
Initialize ensemble prediction: $\hat{\mathbf{q}}_i^{(0)} \leftarrow \mathbf{0}$ for all i
Boosted training:
for round $b \leftarrow 1$ **to** B **do**
  Compute residual targets from $\hat{\mathbf{q}}_i^{(b-1)}$ using (2)
  Initialize per-round feature transform $\mathbb{T}_{\theta_b}$
  Fit base model h_b on $\{(\mathbb{T}_{\theta_b}(\mathbf{z}_i), \mathrm{residual}_i)\}$
  Jointly optimize θ_b and h_b via backpropagation with ℓ_1 regularization on θ_b
  Update ensemble: $\hat{\mathbf{q}}_i^{(b)} \leftarrow \hat{\mathbf{q}}_i^{(b-1)} + h_b(\mathbb{T}_{\theta_b}(\mathbf{z}_i))$
end for
Online inference: For new audio s_i, compute $\mathbf{q}_i = \sum_{b=1}^{B} h_b(\mathbb{T}_{\theta_b}(\mathbb{F}(s_i)))$, apply softmax, select $\hat{j} = \arg\max_j p_{i,j}$

2) FASTT for Non-Differentiable Model Selectors: Tree-based ensemble methods such as gradient-boosted decision trees perform well on structured tabular data and provide strong regularization. However, they are not differentiable with respect to their input features. Gradients cannot be propagated through the discrete tree structure to update the parameters of the feature transformation $\mathbb{T}_\theta$. This creates an optimization challenge since θ and ψ cannot be trained jointly via standard backpropagation.

To enable joint adaptation without requiring differentiability, FASTT uses an alternating optimization strategy [36]. We decompose the joint training problem into two subproblems that are optimized iteratively:

1) *Selector update (fix θ, optimize ψ):* Given the current feature transformation $\mathbb{T}_\theta$, compute the transformed fea-

tures $\mathbf{u}_i = \mathbb{T}_\theta(\mathbf{z}_i)$ and train the tree-based selector on these features.

2) *Transform update (fix ψ, optimize θ):* With the tree structure strictly fixed, update the feature transformation parameters θ.

This alternating procedure is repeated for a fixed number of iterations. The complete training procedure is given in Algorithm 3.

As a representative non-differentiable selector in this path, we utilize XGBoost [37]. Crucially, the selector must minimize the expected-WER objective instead of conventional classification or regression losses. Standard objectives like cross-entropy typically align with categorical targets, whereas FASTT requires minimizing a continuous, sequence-level transcription error (WER) over a probability distribution of distinct STT domains. Therefore, XGBoost is trained with a *custom objective function*. We supply the algorithm with specifically formulated first- and second-order derivatives to seamlessly integrate expected-WER minimization into the tree-building process.

Let $g_{i,j}$ and $h_{i,j}$ denote the first- and second-order derivatives of $\mathcal{L}_i$ with respect to the output logit $q_{i,j}$:

$$g_{i,j} = \frac{\partial \mathcal{L}_i}{\partial q_{i,j}} = p_{i,j}\left(\mathrm{WER}(y_{i,j},\, y_i^{\mathrm{ref}}) - \mathcal{L}_i\right). \tag{5}$$

To reduce computational cost and maintain stable split gains, we use a diagonal approximation to the Hessian:

$$h_{i,j} = \frac{\partial^2 \mathcal{L}_i}{\partial q_{i,j}^2} = p_{i,j}(1 - 2p_{i,j})\left(\mathrm{WER}(y_{i,j},\, y_i^{\mathrm{ref}}) - \mathcal{L}_i\right). \tag{6}$$

We clip negative Hessian values to a small positive constant to ensure stable tree building [37].

Remark 4: The framework is flexible and can accommodate a wide range of learning algorithms for the selection function. In addition to the variants presented here, other architectures such as transformers [38] or alternative supervised learning algorithms could be used without modifying the overall pipeline.

F. Sampling Strategy

Given the selection probabilities $\mathbf{p}_i = (p_{i,1}, \ldots, p_{i,K})$ produced by the selection function, we choose the final expert as

$$\hat{j} = \arg\max_{j \in \{1,\ldots,L\}} p_{i,j}.$$

The selected expert is then executed to produce the final transcription $\hat{y}_i = E_{\hat{j}}(s_i)$ as shown in Fig. 1. Because only a single STT expert is executed, the computational cost of inference remains comparable to that of a single-model system. The selection function itself introduces negligible overhead since it operates on pre-extracted features and involves only lightweight computation relative to the STT models.

IV. Experiments and Results

We evaluate the proposed FASTT framework by using the selector and transformation variants described in Sections III-C and III-E. The experiments are designed to (i) compare the framework variants against external baselines and classical feature selection strategies, (ii) ablate the contribution of different feature transformation and selector choices, and (iii) analyze computational overhead.

Algorithm 3 Offline Training: FASTT for Non-Differentiable Selectors

Inputs:
- $\{(s_1,\, y_1^{\mathrm{ref}}), \ldots, (s_N,\, y_N^{\mathrm{ref}})\}$: Labeled audio clips with reference transcriptions
- $\mathbb{F}$: Feature extraction pipeline
- $\mathcal{E} = \{E_1, \ldots, E_K\}$: Pretrained STT expert pool
- $\mathbb{T}_\theta$: Learnable feature transformation
- T: Number of alternating iterations

Output: Trained feature transformation $\mathbb{T}_\theta$ and selector f_ψ

Offline data preparation:
for $i \leftarrow 1$ **to** N **do**
  Extract features: $\mathbf{z}_i \leftarrow \mathbb{F}(s_i)$
  for each expert $E_j \in \mathcal{E}$ **do**
    Transcribe: $y_{i,j} \leftarrow E_j(s_i)$
    Compute: $\mathrm{WER}_{i,j} \leftarrow \mathrm{WER}(y_{i,j},\, y_i^{\mathrm{ref}})$
  end for
end for
Alternating optimization:
for iteration $t = 1, \ldots, T$ **do**
  Transform features: $\mathbf{u}_i \leftarrow \mathbb{T}_\theta(\mathbf{z}_i)$ for all i
  Compute softmax weights $\mathbf{p}_i$ via (1)
  Compute per-instance gradients $g_{i,j}$ and Hessians $h_{i,j}$ via (5)–(6)
  Selector update: Train XGBoost f_ψ on $\{\mathbf{u}_i\}$ using the custom WER-based targets
  Obtain updated logits: $\mathbf{q}_i \leftarrow f_\psi(\mathbf{u}_i)$
  Transform update: Fix f_ψ; optimize θ using a valid surrogate gradient or continuous approximation
end for
Online inference: For new audio s_i, compute $\mathbf{q}_i = f_\psi(\mathbb{T}_\theta(\mathbb{F}(s_i)))$, apply softmax, select $\hat{j} = \arg\max_j p_{i,j}$

TABLE I
Dataset statistics.

Dataset	Ami	Libri	CV	VoxP.
Total Clips	4091	10500	8200	6400
Features	1672	1672	1672	1672
Train Size	3272	8400	6560	5120
Test Size	819	2100	1640	1280

A. Datasets and Feature Extraction

We construct a heterogeneous evaluation corpus by aggregating clips from four publicly available English speech datasets that span a wide range of acoustic conditions. The corpus includes LibriSpeech [39] (read audiobook speech from *test-clean* and *test-other* splits), Common Voice [40] (crowd-sourced recordings with diverse speaker demographics and recording equipment), VoxPopuli [41] (European Parliament speech with accented English and varying room acoustics), and AMI [42] (meeting recordings using both single distant and individual headset microphones). This corpus is designed so that no single STT model dominates across all conditions, providing a meaningful testbed for adaptive model selection. The dataset statistics are summarized in Table I.

Each clip is represented by a $p = 1672$-dimensional feature vector $\mathbf{z}_i$ composed of the feature groups described in Section III-B. Specifically, the vector concatenates 78 spectral–phonetic features (MFCCs with $\Delta/\Delta\Delta$), 17 prosodic and VAD features, 14 channel and noise features, 8 signal complexity features, 1536 neural embeddings (mean-pooled Wav2Vec2 and HuBERT embeddings, 768 dimensions each), and 19 embedding statistics and metadata features. This high-dimensional representation motivates the learnable feature transformation: directly training a selector on all 1672 features in data-scarce regimes can lead to overfitting and unstable selection.

B. Models and Baselines

1) Individual STT Experts: We employ $L = 3$ pre-trained STT models as the expert pool $\mathcal{E}$: Whisper-large-v3 [4] (a transformer encoder–decoder trained on 680k hours of weakly supervised web audio), Parakeet-TDT-0.6b-v2 (a FastConformer–Transducer model optimized for English ASR with token-and-duration transducer decoding), and Canary-Qwen-2.5b (a multi-task encoder–decoder combining a Fast-Conformer audio encoder with a Qwen-2.5 language model backbone). For each audio clip s_i, all three models are run offline to obtain transcriptions $\{y_{i,j}\}_{j=1}^{3}$ and their associated word error rates against reference transcripts.

2) External Baselines: We compare against non-learned and simple baselines that do not use the FASTT feature transformation pipeline: (i) ROVER [11], which aligns hypotheses from all K models via pivot-based word alignment and applies majority voting; (ii) Random Ensemble, which selects a model uniformly at random for each utterance. We additionally report the Oracle (per-clip selection of the best model) as an upper bound on selection performance.

3) Classical Feature Selection Strategies: To demonstrate the advantage of learned, task-aligned feature transformations over classical approaches, we evaluate four classical feature selection baselines. These methods apply feature selection as a fixed preprocessing step, followed by training a selector on the reduced feature set:

- SelectKBest-MLP: The top k features are selected using univariate statistical tests (ANOVA F-value), and an MLP is trained on the selected features.
- SelectKBest-XGBoost: Same feature selection, followed by an XGBoost selector.
- Wrapper-MLP: Sequential forward selection is used to greedily select a feature subset, with an MLP trained on the result.
- Wrapper-XGBoost: Same wrapper-based selection, followed by an XGBoost selector.

All classical methods use the same expected-WER objective, ensuring a fair comparison with the FASTT framework.

4) FASTT Variants: We evaluate two representative variants within the FASTT framework, corresponding to the differentiable and non-differentiable optimization paths described in Section III-E:

- FASTT-SDT (differentiable path): The Boosted Feature-adaptive Selector with soft decision trees as the base model and diagonal feature gating, trained end-to-end with per-round feature adaptation and ℓ_1-penalized routing.
- FASTT-XGBoost (non-differentiable path): An XGBoost-based selector with learned diagonal feature gating, trained via the alternating optimization procedure described in Section III-E2.

The choice of base model and selector within each path is further studied in the ablation experiments.

TABLE II
HYPERPARAMETER SEARCH SPACES.

Method	Parameter	Search space
Model selector hyperparameters		
XGBoost	Num estimators	{100, 500, 1000}
	Learning rate	{1e-2, 2e-2, 5e-2, 1e-1}
	Max depth	{4, 6, 8, 10}
	Subsample	{0.6, 0.8, 1.0}
	Column subsample	{0.5, 0.8, 1.0}
	Min child weight	{1, 4, 10}
	Reg lambda	{0.1, 1.0, 10.0}
MLP	Learning rate	{1e-4, 5e-4, 1e-3}
	Weight decay	{1e-5, 1e-4, 1e-3}
	Batch size	{512, 1024, 2048}
	Dropout rate	{0.1, 0.2, 0.3}
Soft Decision Tree	Trees per layer	{10, 50, 100}
	Tree depth	{3, 4, 5}
	Lambda (ℓ_1)	{0.01, 0.1, 1.0}
	Temperature	{0.5, 1.0, 2.0}
Feature selection hyperparameters		
SelectKBest	k (num. features)	{50, 100, 200, 500}
Wrapper	Max features	{50, 100, 200}
FASTT	Boosting rounds	{2, 3, 5}
	Learning rate	{1e-4, 5e-4, 1e-3}
	Weight decay	{1e-05, 1e-4, 1e-3}
	Iterations	{3, 5, 10}

C. Training Setup and Evaluation

The dataset is split into 80% training and 20% test sets, stratified by source dataset where possible. Within the training portion, 15% is further reserved for validation to facilitate early stopping and hyperparameter selection. To ensure a fair comparison, all learned methods receive an equal computational budget for hyperparameter tuning. We employ Optuna [43] with a Tree-structured Parzen Estimator (TPE) sampler. The hyperparameter search spaces are given in Table II. The primary evaluation metric is the *mean WER of the selected model* across all test clips:

$$\mathrm{WER}_{\text{method}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{n \in \mathcal{D}_{\text{test}}} \ell\big(y_{i,\,j_i},\ y_i^{\text{ref}}\big)\,, \tag{7}$$

where j_i is the model index selected by the evaluating method.

D. Results and Discussion

1) Transcription Performance: Table III presents the overall and per-dataset WER evaluated on the held-out test set.

TABLE III
TEST WER ACROSS DATASETS.

Method	Ami	Libri.	CV	VoxP.
Individual STT Models				
Whisper-large-v3	0.4050	0.0450	0.1150	0.1750
Parakeet-TDT-0.6b	0.3200	0.0520	0.1050	0.1550
Canary-Qwen-2.5b	0.2880	0.0470	0.1070	0.1350
External Baselines				
Random Ensemble	0.3377	0.0480	0.1090	0.1550
ROVER	0.2780	0.0400	0.0960	0.1300
Classical Feature Selection				
SelectKBest-SDT	0.2700	0.0380	0.0920	0.1280
SelectKBest-XGBoost	0.2680	0.0375	0.0910	0.1270
Wrapper-SDT	0.2650	0.0365	0.0900	0.1260
Wrapper-XGBoost	0.2630	0.0360	0.0890	0.1240
FASTT Framework Variants				
FASTT-XGBoost	0.2540	0.0335	0.0870	0.1200
FASTT-SDT	**0.2480**	**0.0320**	**0.0850**	**0.1150**
Oracle	*0.2320*	*0.0290*	*0.0760*	*0.1060*

TABLE IV
SELECTOR TRAINING TIME (SECONDS) ACROSS DATASETS. STT DECODING AND FEATURE EXTRACTION ARE PERFORMED OFFLINE AND EXCLUDED FROM THESE MEASUREMENTS.

Method	Ami	Libri.	CV	VoxP.
External Baselines				
Random Ensemble	—	—	—	—
ROVER	—	—	—	—
Classical Feature Selection				
SelectKBest-SDT	2.15	8.40	5.95	4.10
SelectKBest-XGBoost	0.41	1.15	0.82	0.65
Wrapper-SDT	129.0	504.0	357.0	246.0
Wrapper-XGBoost	25.1	58.2	42.15	36.80
FASTT Framework Variants				
FASTT-XGBoost	1.25	3.10	2.45	1.95
FASTT-SDT	**10.58**	**26.20**	**20.15**	**16.50**

TABLE V
SELECTOR OVERHEAD: INFERENCE TIME (SECONDS) PER UTTERANCE. STT DECODING AND FEATURE EXTRACTION TIMES ARE EXCLUDED; ONLY THE SELECTOR FORWARD PASS IS MEASURED.

Method	Ami	Libri.	CV	VoxP.
External Baselines				
Random Ensemble	—	—	—	—
ROVER	0.5200	0.4800	0.5000	0.4900
Classical Feature Selection				
SelectKBest-SDT	0.0012	0.0012	0.0012	0.0012
SelectKBest-XGBoost	0.0008	0.0008	0.0008	0.0008
Wrapper-SDT	0.0012	0.0012	0.0012	0.0012
Wrapper-XGBoost	0.0008	0.0008	0.0008	0.0008
FASTT Framework Variants				
FASTT-XGBoost	0.0009	0.0009	0.0009	0.0009
FASTT-SDT	**0.0083**	**0.0083**	**0.0083**	**0.0083**

Several observations emerge from the performance analysis. First, all FASTT framework variants consistently outperform both external baselines and classical feature selection strategies, confirming that the combination of learned feature transformation and task-aligned selection yields a systematic advantage over fixed-preprocessing approaches. Second, the FASTT variants achieve the best overall performance, outperforming the competitor model selector baselines across all datasets, under high dimensionality conditions. This demonstrates the effectiveness of the integrated feature selection within the FASTT architecture. Third, the classical feature selection baselines (SelectKBest, Wrapper) improve over raw-feature selectors but fall short of the FASTT variants, validating the importance of joint optimization. Performance differences between methods are most pronounced on acoustically challenging datasets such as AMI and VoxPopuli, where the benefit of adaptive selection is maximized. Conversely, on cleaner datasets like LibriSpeech, all methods closely approach the oracle performance.

2) Computational Complexity: To evaluate the efficiency of the proposed selection mechanism, we measure the selector training and inference times separately from STT decoding and feature extraction, as reported in Tables IV and V. We emphasize that these measurements reflect only the selector overhead; STT decoding and SSL feature extraction are performed offline and are common to all methods. Because FASTT executes only a single, dynamically chosen STT model during inference, the actual transcription time per utterance strictly matches that of running an individual model. In contrast, ROVER imposes a substantial multiplicative cost by requiring execution of all L experts prior to alignment and voting.

3) Ablation: Feature Transformation and Selector Variants: To isolate the contributions of different components within the FASTT framework, we conduct two ablation studies. Table VI reports the overall WER for varying feature transformation and selector choices.

a) Feature transformation variants: Comparing rows within each selector column, all learned transformations consistently outperform the raw-features baseline, confirming that task-aligned feature adaptation improves selection quality regardless of the selector type. Diagonal gating achieves the best performance in both selector paths, suggesting that feature-wise importance weighting provides the best trade-off between expressiveness and regularization. The nonlinear bottleneck provides moderate improvements over the linear projection but does not surpass diagonal gating, likely because the additional capacity is not justified by the available training data.

b) Selector variants: Comparing columns within each transformation row, we also evaluate different selector models within each optimization path. In the differentiable path, using soft decision trees as the base model in FASTT-SDT outperforms using MLPs, demonstrating the value of the structured boosted architecture. In the non-differentiable path, XGBoost and LightGBM achieve comparable performance, validating that the framework is robust to the specific choice of tree-boosting library. These results support the framework's design as a modular system where components can be selected based on task-specific requirements.

TABLE VI
ABLATION: EFFECT OF FEATURE TRANSFORMATION AND SELECTOR VARIANTS ON TEST WER (MACRO-AVERAGED ACROSS DATASETS).

Method	**Raw**	**Diag.**	**Linear**	**Nonlin.**
Differentiable				
FASTT-SDT	0.1425	**0.1358**	0.1375	0.1365
FASTT-MLP	0.1460	0.1390	0.1405	0.1395
Non-Differentiable				
FASTT-XGBoost	0.1510	**0.1398**	0.1415	0.1408
FASTT-LightGBM	0.1515	0.1402	0.1418	0.1410

4) Interpretability and Feature Importance: A useful property of the FASTT architecture is that the ℓ_1-penalized selection weights yield directly interpretable global feature importance scores. By summing the absolute weights of the routing layers across all depths and boosting rounds, we obtain a feature importance vector $\boldsymbol{\alpha} \in \mathbb{R}^d$. Analysis reveals that neural speech embeddings (specifically Wav2Vec2 and HuBERT dimensions) dominate the top-ranked features, followed by spectral–phonetic descriptors such as higher-order MFCCs and their temporal derivatives. Signal-level quality estimators like SNR and spectral entropy also receive notable decision weight, confirming that the routing mechanism uses embedded signal quality cues to make selection decisions under diverse acoustic conditions.

V. CONCLUSION

REFERENCES

[1] D. Yu and L. Deng, *Automatic Speech Recognition: A Deep Learning Approach*. Signals and Communication Technology, Springer London, 2015. https://doi.org/qvvk.

[2] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*, pp. 12449–12460, 2020. https://doi.org/qvvm.

[3] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021. https://doi.org/grgns2.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 28492–28518, 2023. https://doi.org/gt2rcg.

[5] L. Dong, S. Xu, and B. Xu, "Speech-transformer: A no-recurrence sequence-to-sequence model for speech recognition," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5884–5888, 2018. https://doi.org/gg6672.

[6] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, "Conformer: Convolution-augmented transformer for speech recognition," in *Proceedings of Interspeech*, pp. 5036–5040, 2020. https://doi.org/gjp97z.

[7] A. Graves, "Sequence transduction with recurrent neural networks," in *Proceedings of the Workshop on Representation Learning at the 29th International Conference on Machine Learning (ICML)*, 2012. https://doi.org/qvvn.

[8] J. Li, L. Deng, Y. Gong, and R. Haeb-Umbach, "An overview of noise-robust automatic speech recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 4, pp. 745–777, 2014. https://doi.org/gg8q3b.

[9] J. R. Rice, "The algorithm selection problem," vol. 15 of *Advances in Computers*, pp. 65–118, Elsevier, 1976. https://doi.org/cqnhc2.

[10] C. Giraud-Carrier, P. Brazdil, C. Soares, and R. Vilalta, "Meta-learning," in *Encyclopedia of Data Warehousing and Mining*, pp. 1207–1215, IGI Global Scientific Publishing, 2nd ed., 2009. https://doi.org/dgtwqb.

[11] J. Fiscus, "A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (rover)," in *IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*, pp. 347–354, 1997. https://doi.org/cg5c35.

[12] L. Rabiner and B. H. Juang, *Fundamentals of speech recognition*. Prentice-Hall, Inc., 1993. https://api.semanticscholar.org/CorpusID:268095015.

[13] T. Ko, V. Peddinti, D. Povey, and S. Khudanpur, "Audio augmentation for speech recognition," in *Proceedings of Interspeech*, pp. 3586–3589, 2015. https://doi.org/hbspph.

[14] S. Sun, B. Zhang, L. Xie, and Y. Zhang, "An unsupervised deep domain adaptation approach for robust speech recognition," *Neurocomputing*, vol. 257, pp. 79–87, 2017. https://doi.org/hbsppv.

[15] L. Mangu, E. Brill, and A. Stolcke, "Finding consensus in speech recognition," *Computer Speech & Language*, vol. 14, no. 4, pp. 373–400, 2000. https://doi.org/fbdvbv.

[16] G. Evermann and P. C. Woodland, "Posterior probability decoding, confidence estimation and system combination," in *Proceedings of Speech Transcription Workshop*, 2000. https://api.semanticscholar.org/CorpusID:16725919.

[17] N. Ehling, R. Zens, and H. Ney, "Minimum bayes risk decoding for bleu," in *Proceedings of the 45th Annual Meeting of the ACL on Interactive Poster and Demonstration Sessions*, p. 101, 2007. https://doi.org/c582dt.

[18] J. K. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho, and Y. Bengio, "Attention-based models for speech recognition," in *Proceedings of the 29th International Conference on Neural Information Processing Systems (NeurIPS)*, pp. 577–585, 2015. https://doi.org/qvvp.

[19] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991. https://doi.org/cnsnqg.

[20] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. https://doi.org/g95xv5.

[21] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 1, pp. 1–39, 2022. https://doi.org/qvvq.

[22] Z.-Q. Wang and D. Wang, "A joint training framework for robust automatic speech recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 4, pp. 796—-806, 2016. https://doi.org/qvv3.

[23] J. H. Friedman, "Greedy function approximation: A gradient boosting machine.," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. https://doi.org/fbgj35.

[24] O. Irsoy, O. T. Yildiz, and E. Alpaydin, "Soft decision trees," in *Proceedings of the IEEE 21st International Conference on Pattern Recognition (ICPR)*, pp. 1819–1822, 2012. https://api.semanticscholar.org/CorpusID:9691881.

[25] H. Karaca and S. S. Kozat, "Soft gradient boosting with learnable feature transforms for sequential regression," *IEEE Signal Processing Letters*, vol. 33, pp. 186–190, 2026. https://doi.org/hbsppt.

[26] D. Povey, A. Ghoshal, G. Boulianne, L. Burget, O. Glembek, N. Goel, M. Hannemann, P. Motlicek, Y. Qian, P. Schwarz, J. Silovsky, G. Stemmer, and K. Veselys, "The kaldi speech recognition toolkit," in *IEEE 2011 Workshop on Automatic Speech Recognition and Understanding (ASRU)*, 2011. https://www.danielpovey.com/files/2011_asru_kaldi.pdf.

[27] Y. Li, Z. Zhao, O. Klejch, P. Bell, and C. Lai, "Asr and emotional speech: A word-level investigation of the mutual impact of speech and emotion recognition," in *Proceedings of Interspeech*, pp. 1449–1453, 2023. https://doi.org/hbsppj.

[28] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," in *Readings in Speech Recognition*, pp. 65–74, Morgan Kaufmann Publishers, 1990. https://doi.org/hbsppq.

[29] S. Furui, "Speaker-independent isolated word recognition using dynamic features of speech spectrum," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 34, no. 1, pp. 52–59, 1986. https://doi.org/fb24x3.

[30] J. Sohn, N. S. Kim, and W. Sung, "A statistical model-based voice activity detection," *IEEE Signal Processing Letters*, vol. 6, no. 1, pp. 1–3, 1999. https://doi.org/d64p3z.

[31] B. Schuller and A. Batliner, *Computational Paralinguistics: Emotion, Affect and Personality in Speech and Language Processing*. Wiley Publishing, 1st ed., 2013.

[32] F. Eyben, K. R. Scherer, B. W. Schuller, J. Sundberg, E. André, C. Busso, L. Y. Devillers, J. Epps, P. Laukka, S. S. Narayanan, and K. P. Truong, "The geneva minimalistic acoustic parameter set (gemaps) for voice research and affective computing," *IEEE Transactions on Affective Computing*, vol. 7, no. 2, pp. 190–202, 2016. https://doi.org/f3sfxq.

[33] A. Tripathi, J. Kim, Q. Zhang, H. Lu, and H. Sak, "Transformer transducer: One model unifying streaming and non-streaming speech recognition," *arXiv preprint*, 2020. https://doi.org/qvv8.

[34] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003. https://jmlr.org/papers/volume3/guyon03a/guyon03a.pdf.

[35] R. Kohavi and G. H. John, "Wrappers for feature subset selection," *Artificial intelligence*, vol. 97, no. 1-2, pp. 273–324, 1997. https://doi.org/fngm9z.

[36] D. P. Bertsekas, *Nonlinear Programming*. Athena Scientific, 3rd ed., 2016. http://www.athenasc.com/nonlinbook.html.

[37] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794, 2016. https://doi.org/gdp84q.

[38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31th International Conference on Neural Information Processing Systems (NeurIPS)*, 2017. https://doi.org/gpnmtv.

[39] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An asr corpus based on public domain audio books," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5206–5210, 2015. https://doi.org/gfv84w.

[40] R. Ardila, M. Branson, K. Davis, M. Kohler, J. Meyer, M. Henretty, R. Morais, L. Saunders, F. Tyers, and G. Weber, "Common voice: A massively-multilingual speech corpus," in *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, pp. 4218–4222, 2020. https://doi.org/qvvt.

[41] C. Wang, M. Riviere, A. Lee, A. Wu, C. Talnikar, D. Haziza, M. Williamson, J. Pino, and E. Dupoux, "Voxpopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, pp. 993–1003, 2021. https://doi.org/gm6jcn.

[42] I. Mccowan, J. Carletta, W. Kraaij, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos, M. Kronenthal, G. Lathoud, M. Lincoln, A. Lisowska Masson, W. Post, D. Reidsma, and P. Wellner, "The ami meeting corpus," in *Proceedings of the International Conference on Methods and Techniques in Behavioral Research (MB)*, pp. 1–4, 2005. https://researchgate.net/publication/228341280.

[43] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM International Conference on Knowledge Discovery & Data Mining (KDD)*, pp. 2623–2631, 2019. https://doi.org/gf7mzz.

[44] R. E. Schapire and Y. Freund, *Boosting: Foundations and Algorithms*. The MIT Press, 2012. https://doi.org/hbsppn.

[45] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. https://doi.org/gdhpjv.

[46] S. Ghadimi and G. Lan, "Stochastic first- and zeroth-order methods for nonconvex stochastic programming," *SIAM Journal on Optimization*, vol. 23, no. 4, pp. 2341–2368, 2013. https://doi.org/ggmm89.

[47] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019. https://doi.org/gqp96f.

Appendix A
Convergence Analysis

We provide convergence guarantees for the selector optimization procedures presented in Section III-E. Specifically, we show that under mild regularity conditions, both training procedures converge to a first-order stationary point of the training objective.

Remark 5: These results characterize the convergence behavior of the *selector optimization* under fixed or jointly updated feature transformations. Formal convergence guarantees for the end-to-end joint system (feature transform + selector) in the non-differentiable alternating optimization case remain an open question and are left for future investigation.

The proofs hold for any smooth loss with bounded curvature, consistent with the framework's loss-agnostic design. Our convergence analysis focuses on the optimization of the selector parameters for a fixed feature transformation. In the full framework, the transformation parameters are updated either jointly (for differentiable selectors) or through alternating optimization (for tree-based selectors), which preserves the stationary-point guarantees for the selector subproblem. The same convergence guarantees apply to alternative admissible objectives (e.g., cross-entropy, MSE, or other smooth sequence-level surrogates) that satisfy the smoothness assumptions used in the analysis.

A. Convergence of the XGBoost-Based Selector

a) Preconditions (A1–A3):

A1. *Bounded cost.* Each of L candidate STT models E_j produces a transcription $y_{i,j}$ for input s_i, and the associated per-model cost is $c_{i,j} := \mathrm{WER}(y_{i,j}, y_i^{\mathrm{ref}})$. We assume two uniform bounds:

$$0 \;\le\; c_{i,j} \;\le\; B_{\max} := \max_{i,j} c_{i,j},$$
$$|c_{n,i} - c_{n,j}| \;\le\; B_{\mathrm{rng}} := \max_{n} \max_{i,j} |c_{n,i} - c_{n,j}|.$$

The second derivative term is $h_{i,j} = p_{i,j}(1-2p_{i,j})(c_{i,j} - L_n)$, where $L_n = \sum_j w_{n,j}\, c_{n,j}$ is a convex combination of the costs. Thus

$$c_{i,j} - L_n = \sum_{j \neq m} w_{n,j}(c_{i,j} - c_{n,j}),$$
$$|c_{i,j} - L_n| \le (1 - p_{i,j})\, B_{\mathrm{rng}}.$$

Therefore, $|h_{i,j}| \;\le\; B_{\mathrm{rng}}\, p_{i,j}(1 - p_{i,j})\, |1 - 2w_{i,j}| \;\le\; \frac{\sqrt{3}}{18}\, B_{\mathrm{rng}} \;<\; \frac{B_{\mathrm{rng}}}{8}$. After clipping $h_{i,j}$ below by $\varepsilon > 0$, all diagonal entries of the approximate Hessian H^* lie in $[\varepsilon,\, B_{\mathrm{rng}}/8]$, giving $\varepsilon I \preceq H^* \preceq \frac{B_{\mathrm{rng}}}{8}\, I$. We also assume that the overall loss L is L_g-smooth, i.e., its gradient is Lipschitz with constant $L_g > 0$.

A2. *Weak-learner edge.* Let $z^{(t)} \in \mathbb{R}^{N \times K}$ be the prediction matrix at boosting iteration t, and write $g_t := \nabla_z L(z)\big|_{z=z^{(t)}} \in \mathbb{R}^{N \times K}$. Let f_t be the depth-≥ 1 regression tree obtained by least-squares fitting to the pseudo-targets $-g_t$, and define the update $z^{(t+1)} = z^{(t)} + \eta\, f_t$ with $\eta > 0$. We assume that f_t satisfies the *edge condition* $\langle -g_t, f_t \rangle \ge \rho\, \|g_t\|_2^2$, where $\|\cdot\|_2$ denotes the empirical ℓ_2 norm over (n, m) pairs, and the constant $\rho > 0$ depends on the tree depth and regularization [44].

A3. *Quadratic fit bound.* The regularized least-squares fit in XGBoost satisfies $\|f_t\|_2^2 \;\le\; \lambda^{-1} \|g_t\|_2^2$, where $\lambda > 0$ is the ℓ_2 regularization coefficient in the leaf-weight formula [37].

b) Descent lemma: Let $L_t := L(z^{(t)})$. By the smoothness assumption (A1), $\nabla_z L$ is L_g-Lipschitz. Using a second-order Taylor expansion:

$$L_{t+1} \leq L_t - \eta\rho\|g_t\|_2^2 + \tfrac{1}{2}\eta^2 L_g \|f_t\|_2^2.$$

Substituting A3:

$$L_{t+1} \leq L_t - \eta\left(\rho - \tfrac{\eta L_g}{2\lambda}\right)\|g_t\|_2^2. \tag{6}$$

Define $\gamma := \eta\rho - \frac{\eta^2 L_g}{2\lambda}$. Choosing $\eta < \frac{2\rho\lambda}{L_g}$ ensures $\gamma > 0$ and hence $L_{t+1} \leq L_t - \gamma\|g_t\|_2^2$.

c) Convergence rate: Summing (6) over t gives:

$$\sum_{t=0}^{T-1} \|g_t\|_2^2 \leq \frac{L_0}{\gamma} \quad \Rightarrow \quad \min_{0\leq t<T} \|g_t\|_2^2 \leq \frac{L_0}{\gamma T},$$

ensuring a convergence rate of $O(1/T)$ toward a first-order stationary point. Since L is non-convex, this does not imply global optimality.

B. Convergence of the Neural Selector (MLP / FASTT-SDT)

a) Preconditions (A1–A3): For $L(\theta)$ defined in (3):

A1. *Bounded loss:* $0 \leq L(\theta) \leq B := \max_{i,j} c_{i,j}$.

A2. *L_g-smooth gradient:* $\|\nabla L(\theta) - \nabla L(\theta')\| \leq L_g \|\theta - \theta'\|$ for all θ, θ'.

A3. *Bounded stochastic-gradient variance:* $\mathbb{E}\big\|g_t - \nabla L(\theta_t)\big\|^2 \leq \sigma^2$.

Let $L^\star := \inf_\theta L(\theta)$. Under A1–A3, the standard assumptions in the non-convex analyses of Bottou–Curtis–Nocedal [45] and Ghadimi–Lan [46] are satisfied.

Remark 6: Our analysis assumes $L(\theta)$ is L_g-smooth, which holds when using smooth activations such as GELU or Softplus. In practice, we train with AdamW [47] for efficiency, though the convergence proof is stated for (stochastic) gradient descent.

b) One-step descent lemma: For the full-batch update $\theta_{t+1} = \theta_t - \eta\nabla L(\theta_t)$ with any fixed step size $0 < \eta \leq 1/L_g$:

$$L(\theta_{t+1}) \leq L(\theta_t) - \left(\eta - \tfrac{L_g}{2}\eta^2\right)\|\nabla L(\theta_t)\|^2. \tag{7}$$

c) Full-batch convergence: Summing (7) for $t = 0, \ldots, T-1$:

$$\min_{0\leq t<T} \|\nabla L(\theta_t)\|^2 \leq \frac{L(\theta_0) - L^\star}{T\,\eta\big(1 - \frac{1}{2}\eta L_g\big)} = \mathcal{O}\big(T^{-1}\big). \tag{8}$$

d) Mini-batch SGD convergence: With $\eta_t = \eta_0/\sqrt{t+1}$, $\eta_0 \leq 1/L_g$, and using A3:

$$\mathbb{E}\Big[\min_{0\leq t<T} \|\nabla L(\theta_t)\|^2\Big] \leq \frac{2\big(L(\theta_0) - L^\star\big) + \eta_0 L_g \sigma^2 \log T}{\eta_0\sqrt{T}} = \mathcal{O}\big(T^{-1/2}\big), \tag{9}$$

where σ^2 is the stochastic-gradient variance bound from A3. Because L is non-convex, (8)–(9) guarantee convergence only to a first-order stationary point, which is sufficient for training neural selectors [46].